



# Python



Python ofrece varios modelos de ejecución concurrente: hilos, procesos y concurrencia asíncrona. Sin embargo, el comportamiento real depende de un elemento clave del intérprete: el **Global Interpreter Lock (GIL)**.

## Explicación

En Python:

- **Sí existe multithreading** (múltiples hilos).
- **NO existe paralelismo real entre hilos CPU-bound** debido al GIL.
- Sí es posible conseguir **paralelismo real** mediante **multiprocessing** o código nativo que libere el GIL.

## El Global Interpreter Lock

El **Global Interpreter Lock** garantiza que **solo un hilo pueda ejecutar bytecode de Python a la vez** (debido a decisiones de diseño del lenguaje de programación).

Esto implica:

- Con hilos: **conurrencia**, pero no paralelismo real.
- Con tareas CPU-bound: **speedup máximo = 1**.
- Con tareas I/O-bound: puede haber mejoras reales, porque el hilo que espera I/O libera el GIL.
- Con procesos: cada proceso tiene su propio GIL → **paralelismo verdadero**.

Esto significa que Python NO puede lograr  $\text{speedup} > 1$  con hilos en tareas CPU-bound.

Pero Sí puede conseguir speedup real mediante:

- Multiprocessing
- Extensiones C que liberan el GIL (NumPy, TensorFlow...)
- Tareas I/O-bound

## Hilos

Los hilos se crean con `threading.Thread`.

Al instanciar un Thread, el hilo **se lanza automáticamente**.

```
from threading import Thread
```

```
def trabajo():  
    print("Hola desde un hilo")
```

```
t = Thread(target=trabajo)  
t.start()  
t.join()
```

## Cerrosos

Python ofrece cerrosos para proteger secciones críticas:

```
from threading import Lock
```

```
contador = 0  
lock = Lock()
```

```
def incrementar():  
    global contador  
    for _ in range(100000):
```

```
with lock: # sección crítica
    contador += 1
```

## Monitores

Python **no** tiene monitores nativos, pero **sí permite implementarlos** con:

- un `Lock` (el cerrojo interno del monitor),
- un `Condition` (la variable de condición con su wait-set).

```
import threading

condition = threading.Condition()
lista = []

# Consumidor

def consumidor():
    with condition:
        while not lista:
            condition.wait() # entra en el wait-set
        valor = lista.pop(0)
        print("Consumido", valor)

# Productor

def productor():
    with condition:
        lista.append(10)
        condition.notify() # despierta a un hilo esperando
```

## Ejecutores

```
from concurrent.futures import ThreadPoolExecutor

def tarea(n):
    return n * 2

with ThreadPoolExecutor(max_workers=4) as pool:
    resultados = pool.map(tarea, [1,2,3,4])
    print(list(resultados))
```

Su utilidad real viene para I/O-bound, no para CPU, ya que el GIL nos bloquea el paralelismo a nivel de CPU para que solo haya 1 hilo bytecodeando a la vez.

## Multiprocessing

Como el GIL impide paralelismo CPU con hilos, Python usa **multiprocessing** para verdaderos cálculos paralelos. Podemos conseguir paralelismo REAL a nivel de proceso, no a nivel de hilo (a menos que hagamos lo que hemos descrito previamente para liberar el GIL).

Cada proceso:

- tiene su propio intérprete Python,
- tiene su propio GIL,
- puede ejecutar en un core distinto.

```
from multiprocessing import Process

def cuadrado(n):
    print(n * n)
```

```
p = Process(target=cuadrado, args=(5,))
p.start()
p.join()
```

## Comunicación entre procesos

Los procesos NO COMPARTEN MEMORIA, así que se comunican de distintas formas:

**Pipes** (similar al paso de mensajes)

```
from multiprocessing import Pipe, Process

def hijo(conn):
    conn.send("hola padre")
    conn.close()

padre, hijo_conn = Pipe()
p = Process(target=hijo, args=(hijo_conn,))
p.start()
print(padre.recv()) # recibe el mensaje
p.join()
```

## Colas FIFO compartidas

```
from multiprocessing import Queue, Process

cola = Queue()

def productor():
    cola.put(42)
```

```
def consumidor():
    valor = cola.get()
    print("Consumido", valor)

p1 = Process(target=productor)
p2 = Process(target=consumidor)
p1.start(); p2.start()
p1.join(); p2.join()
```

## Sincronización

### Cerrosjos

```
from multiprocessing import Lock
lock = Lock()
```

### Barrier

```
from multiprocessing import Barrier, Process

b = Barrier(3)

def trabajo():
    print("antes")
    b.wait() # todos paran aquí
    print("después")
```

### Condition

```
from multiprocessing import Condition
cond = Condition()
```